

DIGITAL FORENSICS & NETWORK TRAFFIC INVESTIGATION REPORT

Technical Analysis of Plaintext Credential Exposure via HTTP Capture

Prepared By:	Osandi	Date:	July 9, 2026
Environment:	Virtual Laboratory (VirtualBox)	Status:	Completed / Verified

1. EXECUTIVE SUMMARY

This technical report documents a controlled network security investigation focused on analyzing credential exposure over unencrypted communication channels. The primary objective was to observe how user credentials (usernames and passwords) are transmitted over the standard Hypertext Transfer Protocol (HTTP) and to demonstrate the ease with which an inline attacker or a Security Operations Center (SOC) analyst can intercept sensitive data using passive network packet analysis tools. Using Wireshark as the packet analyzer, live network traffic was captured on a shared local subnet, isolated, and dissected. The captured data conclusively revealed that the authentication parameters were broadcasted in clear text, posing a critical data security and confidentiality risk.

2. LAB ENVIRONMENT SETUP & TOPOLOGY

The investigation was simulated inside an isolated virtual laboratory environment to guarantee strict containment and safety:

- **Attacker / Analysis Node:** Kali Linux Virtual Machine (IP: 192.168.8.142) acting as the capture platform and running a standalone Python3 HTTP server on Port 80.
- **Victim Node:** Windows Host Workstation (IP: 192.168.8.135) interacting via a standard modern web browser.
- **Network Infrastructure:** VirtualBox Bridged Networking, allowing both endpoints to communicate directly over the same physical local IP subnet (192.168.8.0/24).

3. INCIDENT SIMULATION & ANALYSIS METHODOLOGY

The investigation proceeded through three systematic, sequential phases:

Phase 1: Packet Capture Initialization

Live packet sniffing was activated on the Kali Linux eth0 interface using Wireshark to record all raw Ethernet frames traversing the shared collision domain.

Phase 2: Data Generation (The Phishing Vector)

A lightweight web utility was initiated on the Kali machine using the command `sudo python3 -m http.server 80`. The victim browsed to a targeted, unencrypted URL hosted on the server, submitting dynamic credential values directly into the web application via an HTTP GET query string.

Phase 3: Traffic Filtering

Out of 1,070 total captured packets, a display filter of `http` was applied to strip away background noise (such as unrelated broadcast protocols, ICMPv6, and raw TCP control handshakes) and cleanly isolate application-layer traffic.

4. INVESTIGATION FINDINGS & EVIDENCE

Traffic Overview

Upon applying the `http` filter, a sequence of web requests and responses was discovered between the victim and the target server, detailing the entire transaction lifecycle:

Packet No.	Source IP	Destination IP	Protocol	Info Summary
640	192.168.8.135	192.168.8.142	HTTP	<code>GET / HTTP/1.1</code> (Initial directory listing request)
646	192.168.8.142	192.168.8.135	HTTP	<code>HTTP/1.0 200 OK (text/html)</code> (Directory successfully loaded)
954	192.168.8.135	192.168.8.142	HTTP	<code>GET /login.html?username=osadi_secure&password=mysecretpass123 HTTP/1.1</code>
957	192.168.8.142	192.168.8.135	HTTP	<code>HTTP/1.0 404 File not found</code> (Server acknowledgment response)

Deep Packet Inspection (DPI)

As captured in **Packet No. 954**, the victim (`192.168.8.135`) transmitted an explicit outbound request containing raw authentication data directly to the suspicious web server (`192.168.8.142`).

Because standard HTTP does not implement cryptographic layers, the transmission string was fully exposed in plain text within the Uniform Resource Identifier (URI) parameters:

- **Captured Parameter 1 (Username):** `osadi_secure`
- **Captured Parameter 2 (Password):** `mysecretpass123`

Although the Python server responded with an HTTP 404 File not found error code (Packet No. 957), the credential harvesting objective was entirely successful since the plaintext string hit the wire and was permanently recorded by the packet capture tool.

Visual Evidence (Wireshark Capture)

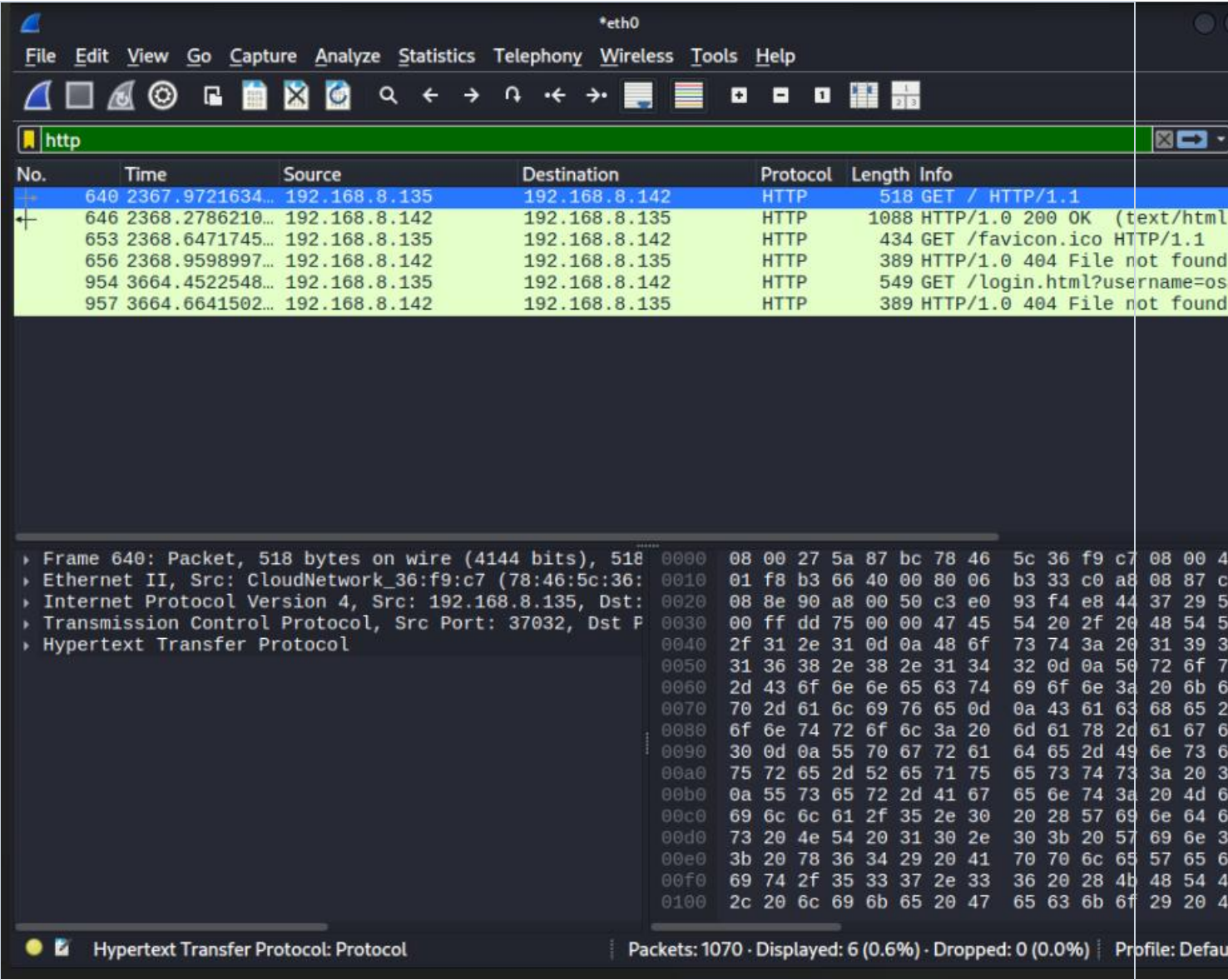


Figure 1: Wireshark packet capture analysis displaying filtered HTTP traffic and plaintext parameters in Packet 954.

5. STRATEGIC REMEDIATION & MITIGATION

To secure the network infrastructure against this specific vulnerability, the following mitigations must be deployed immediately:

1. **Mandate Transport Layer Security (TLS/HTTPS):** Enforce an absolute migration from HTTP to HTTPS. Implement HTTP Strict Transport Security (HSTS) headers to force browsers to interact with web servers exclusively over secure channels.
2. **Transition to Secure POST Methods:** Authentication data should never be appended to a URL using GET requests (as it leaves strings exposed in browser histories, proxies, and network logs). Use secure HTTP POST requests wrapped inside an encrypted TLS tunnel.
3. **Deploy Multi-Factor Authentication (MFA):** Establish robust MFA policies across all public and internal authentication endpoints to ensure that stolen plaintext strings alone cannot grant unauthorized access.
4. **Network Access Control:** Implement switch-level protections such as DHCP Snooping and Dynamic ARP Inspection (DAI) to reduce the risks of local Man-in-the-Middle (MitM) sniffing on physical local segments.